AIClaimsAssist

Week 3 Project Documentation | The Gen Academy — Mastering Agentic AI | November 2026

1. Overview and problem

AIClaimsAssist is a local claims investigation prototype that helps an adjuster review claim information, missing documents, evidence observations, policy passages, and vehicle identification details. It organizes findings for human review; it does not approve, deny, or settle claims.

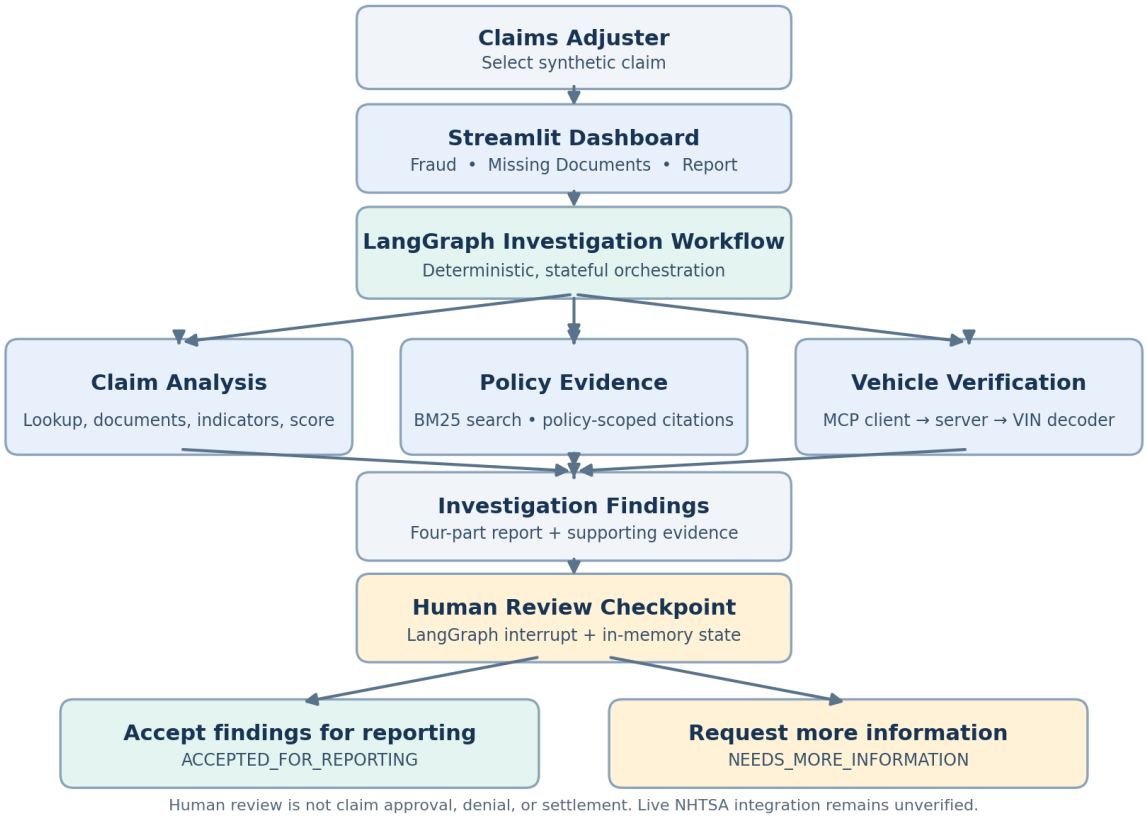
Claims investigation requires reconciling notes, policies, supporting documents, and potential discrepancies. The project asks whether a structured tool-based workflow can make this review clearer, more traceable, and more consistent.

2. Implemented solution

- 20 synthetic Motor/Auto claims with notes, document inventory and evidence observations.
- Claim lookup and status/loss-dependent missing-document analysis.
- Evidence indicators separated into confirmed, unverified, incomplete-data, and disproved states.
- Deterministic 0–100 investigation-priority score from confirmed weighted indicators; not a fraud probability.
- Local BM25 policy search with policy-ID isolation and source citations.
- MCP stdio server/client VIN decoder integration and claim-to-vehicle comparison.
- LangGraph stateful investigation workflow with interrupt/resume human review.
- Streamlit dashboard with Fraud, Missing documents, and Report tabs.

3. Architecture

Figure 1. AIClaimsAssist solution architecture



Streamlit dashboard → LangGraph investigation workflow → claim lookup, missing-document analysis, evidence evaluation and scoring → policy BM25 retrieval and MCP VIN verification → structured report → human-review interrupt → resumed review status.

The MCP tool calls a VIN decoder designed for NHTSA vPIC. Live NHTSA verification was not established in final testing; offline MCP responses were mocked. The workflow follows a defined sequence rather than model-directed tool selection.

4. Technology stack

Component	Technology / role
Orchestration	Python, LangGraph; LangChain packages installed
Tool protocol	MCP Python SDK over stdio
Policy retrieval	Local BM25 over approved policy chunks
Interface	Streamlit
Data	Synthetic JSON claims and policy documents
Testing	pytest, Streamlit AppTest

5. Investigation report

Fraud Risk Score

Displays the investigation-priority score, breakdown, and caution that it is not a fraud probability.

Fraud Description

Separates confirmed discrepancies from unverified observations, with source references and review flags.

Missing Documents

Lists outstanding required and applicable conditional documents with reasons.

Claim Summary and Notes

Shows claim and policy IDs, status, loss facts, amount, summary, and chronological notes.

Supporting information includes vehicle verification and policy citations. The report is constructed from existing graph state without rerunning investigation tools.

6. Human-in-the-loop

LangGraph pauses at a human-review checkpoint. ACCEPT_REPORT resumes with ACCEPTED_FOR_REPORTING; REQUEST_MORE_INFO resumes with NEEDS_MORE_INFORMATION. These are report-review outcomes only, not claim decisions. Checkpoints and review decisions are in memory for this prototype; review notes are not required.

7. Running locally

From C:\ShilpiStuff\Projects\week3, with the existing virtual environment and project data:

.\.venv\Scripts\python.exe -m streamlit run app.py

Run tests with:

.\.venv\Scripts\python.exe -m pytest tests -q

Keep any .env credentials local and ignored by Git. Do not commit API keys or the virtual environment.

8. Final offline validation

Check	Result
Automated regression tests	83 passed
Synthetic claims investigated	20/20
Claims paused for human review	20/20

Policy isolation failures	0
Invalid risk scores	0
Review decisions	Both passed; no tool reruns on resume
Report serialization and Streamlit checks	Passed

Validation used mocked MCP responses and did not make live Nebius, NHTSA, or Pinecone requests. Claim data was not modified.

9. Safety and limitations

- All claims and observations are synthetic; physical claim evidence is limited in the fixture.
- LLM-driven tool selection was not implemented; Nebius Gemma tool calling remains unverified.
- Retrieval is local BM25, not Pinecone-backed Hybrid RAG.
- Live NHTSA verification is unverified; failure is reported as UNABLE_TO_VERIFY.
- Human-review checkpointing and review decisions are not persisted in a database.
- An investigation-priority score is not a fraud finding or an automated coverage decision.

10. Conclusion

AIClaimsAssist demonstrates how a LangGraph workflow, specialized investigation tools, MCP integration, evidence traceability, and human review can support claims adjusters. The next priorities would be a policy-approved LLM tool-calling integration, richer evidence fixtures, hybrid retrieval, and durable audit history.